

Deep Learning Applications

Exam Report

Edoardo Canti - 2025/2026

1. Presentations

The following table shows for every presentation the chosen paper title, its arxiv link, the link to the presentation itself and, when necessary, the link to the video (as accorded in class for peer assessments).

Links to presentations and **videos** redirects to my personal **Google Drive** University account and each single element should have open access without requiring any confirmation by the owner.

Please if something is not accessible as it should be contact me (trials with incognito mode worked with the links).

Purpose	Title	Link to the article	Link to the presentation	has video
Workshop I	Neural Collapse by Design: Learning Class Prototypes on the Hypersphere	https://arxiv.org/pdf/2605.20302v1	https://drive.google.com/file/d/1Mt4XP26dYodMLF3dF-3Xa3sGlXh86LKS/view?usp=sharing	https://drive.google.com/file/d/1XQcR_H9o3QzD2uOAsx-gg9AU-Wf8Z3EN/view?usp=sharing
Workshop II	Missingness Bias Calibration in Feature Attribution Explanations	https://arxiv.org/pdf/2603.04831	https://drive.google.com/file/d/1Fp2G_W-7hZ19JK0nQ0rRDbgRIOeZJGOW/view?usp=sharing	https://drive.google.com/file/d/1mSLxAx0C1pJ0QAT1pNAyHB-3fQk-BDys/view?usp=sharing
Argomento di Approfondimento	VL-JEPA: Joint Embedding Predictive Architecture for Vision-language	https://arxiv.org/pdf/2512.10942	https://drive.google.com/file/d/1O-0N5Plvc7Z6jGKZX5Rktn-VSEccKiL/view?usp=sharing	No

2. Laboratories

All Laboratories have been deployed using a single repo in the personal **GitHub** page: <https://github.com/EdoardoCanti/DeepLearningApplications>

The entry point is a **general directory** with a general readme file and **three distinct subdirectories**, each representing a single laboratory.

Chosen Laboratories have been:

1. **From Pixels to Semantics**
2. **The Transformative Transformer**
3. **Getting Up to Speed with Reinforcement Learning**

Each of the previous mentioned directory **has a ReadMe** written with the following purposes:

- Files organization (where you can find what and the purpose of single files/directories)
- An high level summary of the tasks that were asked to solve
- Execution instructions, with personal advices on how to execute (and if needed links to trained models to be downloaded)
- Personal notes and explanation on how the exercise has been approached and solved.

For what concerns explanations and solutions, when possible those have been written inside the jupyter notebooks themselves rather than in the README (but it should be written if that's the case).

IMPORTANT NOTES:

requirements.txt files for conda environments (conda was used) were deployed if and only if the involved environment is different with respect to the original provided in moodle for flipped lectures.

Spoiler: only for Laboratory2 -The Transformative Transformer, there is a different env.

2.1 Disclaimers

In these laboratories some changes or “adds” with respect to the original path have been applied (not huge differences of course). I hope the final results are aligned enough with the original requests.

Changes or “adds” have been essentially driven by two factors:

- personal curiosity about frameworks that were used rarely (Optuna).
- A personal interest in the branch of Representation Learning (that also drove the choice of Argomento di Approfondimento and Workshop I)

Throughout the code can be found notes assessing the usage of LLMs. Gemini online free-version has been used when needed.

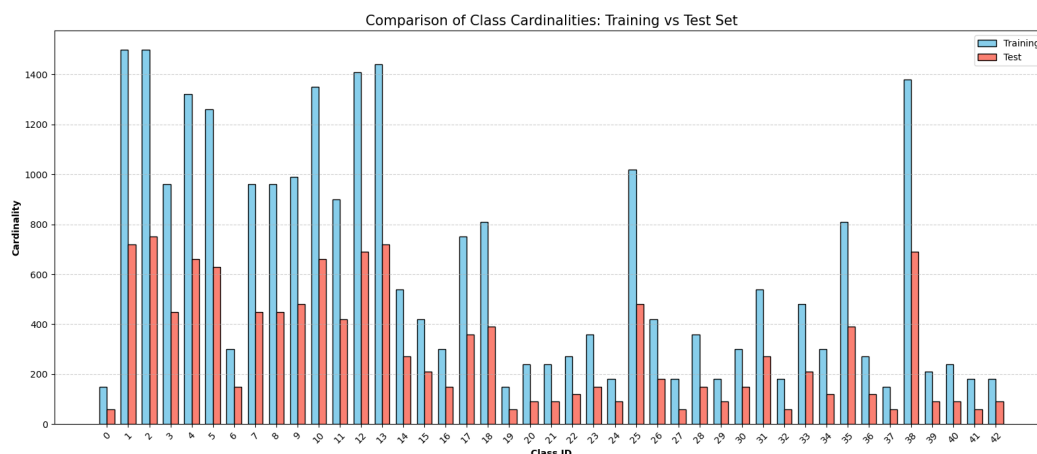
Also throughout the code (whether in form of notebook or py file) can be found additional explanations that were written primarily as personal notes and then revised in order to be an explanation for the reader. Essentially this notes should be read not only as personal reflections, interpretations, analysis and eventually solutions but also as findings about the usage of some constructs for instance in LAB2 I struggled a bit while reproducing flipped lectures code, probably this is for the usage of FAST huggingface components instead of Vanilla); also there are notes reporting links that can be considered as sources of what has been developed.

3. Labs Solutions Explanations

The following section is a summary of the findings during Laboratories solutions.

Laboratory 1: From Pixels to Semantics

An Exploratory Data Analysis has been conducted essentially because the GTSRB Dataset is deployed with an A Priori data division into Training and Test set. Since in a traditional Deep Learning pipeline Training, Validation and Test sets are involved and sometimes “problems” can arise from a missing data stratification, this exploratory analysis has been conducted.



Results

showed that the number of samples on each class, for both Training set and Test set are pretty balanced. This result had the consequence to identify groups of classes based on their representation levels (Highly represented, Medium represented and low represented). These groups were used throughout the rest of the Laboratory in order to comments results.

After this first part A stable Baseline has been implemented (exactly as in the legacy code of Flipped Lecture). Remember that here the goal was to use the RESNET-18 backbone as a feature extactor, freezing it and evaluate the performances of an SVC baseline over the RESNET-18 GTSRB extracted features. As reported in notebook LAB1_exercise1.2_1.3-BASELINE.ipynb, results seems to confirm EDA findings: on average, on high represented classes, a not GTSRB fine tuned Resnet backbone with an SVC classifier head performs slightly better than others.

This baseline was than fine tuned applying several experiments, for implementing a from scratch HyperParameters optimization and by freezing the first 2 layers of resnet and applying two different classification heads: a Linear and an MLP (MLP was just for completness wanted to try WandB). For this an MLP class has been written in utils.py supporting different types of weights initialization and activation functions, but to be honest the experiments have been ran only with standard relu and He Uniform weight initialization.

With Linear Head the best configurations of hypeparameters have been 30 epochs, learning rate of 0.001 and a batch size of 128. With the following training and validation losses trends. On these settings the model reached a test accuracy of 0.947. The following image is a screenshot from the original notebook reporting analysis.

Experiment 1 - Analysis of results

Best result: EP30_LR0.001_BS128

The best result has been obtained using 30 training epochs, with an Adam Optimizer equipped with a learning rate of 0.001 and a batch size of 128.

The model reached a test accuracy of 0.947.

The **Classification report** shows some coherent aspects wrt the *Exploratory Data Analysis* reported in LAB1_exercise1-EDA.ipynb:

- The model performs very well on the so called: **Highly Represented Classes** (2,1,13,12,38,10,4,5)
- The **weighted avg** is slightly better than **macro**, this means that the model performs well on the most represented classes, while low represented classes tends to lower the macro average (classification report of classes: 19, 27, 28, 29, 30...)
- The **gap** between **weighted avg** **macro avg** also confirms what has been stated during flipped lecture:

"we can treat the problem as a not **too** imbalanced one" By taking a look at the **loss_chart.png** we can say that the general trend is increasing in performances, also if we can identify a non-negligible presence of spikes.

By the way, since the spikes seems to be counterbalanced by local minima, we can conclude that the model is not going to overfit (at least at epoch 30).

The conclusion is that a Fine-Tuned ResNet-18 with *Linear layer* head, is enough to achieve *good* performances. this suggest that the backbone **ResNet18** has a representation power, strong enough, to make a Linear model a good choice.

Also, the fact that the ResNet-18 was fine-tuned, by freezing the first two layers, I think shows the following fact; as known the first layers of a CNN can learn simple filters, while successive layers tend to learn more complex patterns, probably the original data on which the first two layers of ResNet18 were trained, has been enough to learn patterns that, *arriving at the Linear model head* were already learnt.

The final exercise of LAB1 has been the one regarding embeddings. The requested Nearest Mean Retriever was implemented but before of diving into that some experiments about manifold explorations were conducted.

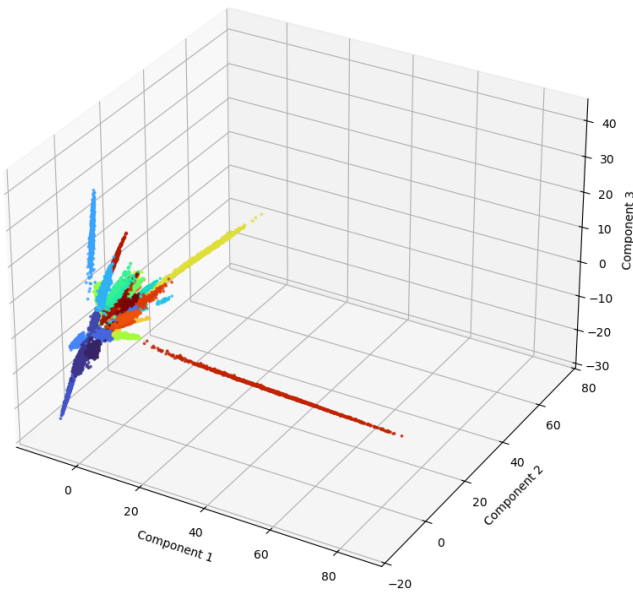
First of all, ResNet18 were used as Features Extractor. Passing an input to a features extractor means to receive as output the internal representation of the input, i.e: how the feature extractor has learnt during its training to represent **that** input. When we apply fine tuning we are asking the model to *adapt* its prior representation space to new data it didn't see before, the hope in this case is that it will not forget what was learnt during pretraining phase. It reasonable to assume that the model will represent in a different manner the new data when it is fine tuned and when it isn't. This is the idea that drove this exercise.

Also we should consider another thing: especially when dealing with a nearest mean classifier we would have the interest that all representations lies on the same manifold. To achieve this those features were also projected into the unit hypersphere; this is because ideally the distance (especially in terms of clusters) could be not a proper measure of the information represented by the feature. Imagine for instance two features with the same directions, but one is incredibly larger in magnitude with respect to another, without normalization we could consider them as very different.

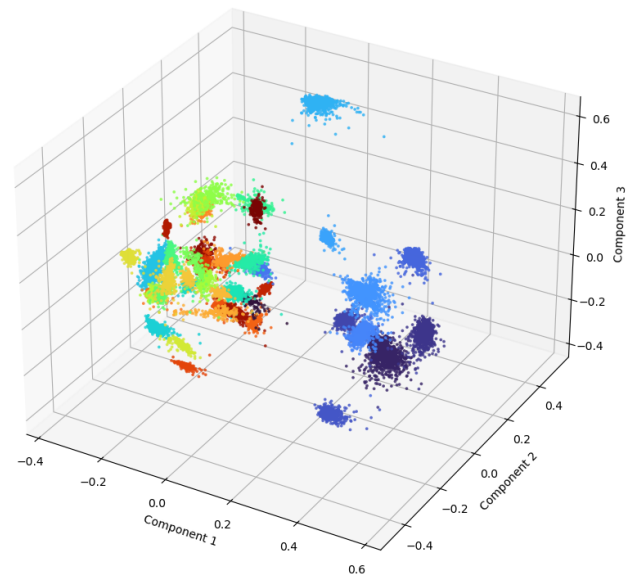
Conducted evaluations using Silhouette's Coefficient showed the difference in clusters coherence and separation using Vanilla Resnet, Fine Tuned Resnet both of them in the not normalized space and normalized space.

What has been observed has been that fine tuned versions of resnet isolates different clusters instead of maintaining overlapping point clouds and normalization brings an increment in Cohesion and Separations of cluster in both cases: Vanilla and Fine Tuned.

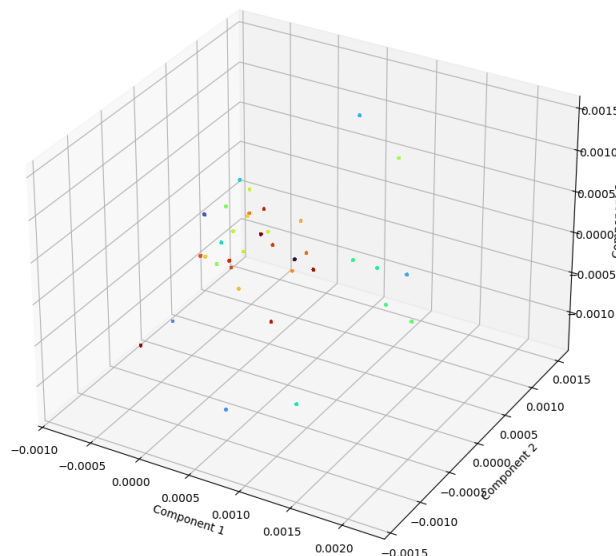
Please also consider that ResNet18 representation space (latent space) has a 512 dimensions, how could we take a look at those manifolds? An idea was to use Principal Component Analysis to reduce dimensions, the drawback of this method is that it based on the data variance. Findings are somehow more interesting in the case of not normalized, but fine tuned, embeddings instead of normalized when (that were actually expected). Because of this another dimensionality reduction (Diffusion Maps, found on "Manifold Learning: what, how and why?") was tested and results are definitely more indicatively for the fine tuning results.



PCA - Fine Tuned ResNet18 NOT- Normalized Embeddings
(GTSRB and first 2 layers frozen)



PCA - Fine Tuned ResNet18 Normalized Embeddings
(GTSRB and first 2 layers frozen)



Spectral Clustering - Fine Tuned ResNet18 Normalized
Embeddings (GTSRB and first 2 layers frozen)

Nearest Mean Classifier performances showed an accuracy of 95.69%. Reporting a personal conclusion on this that can be found in the notebook:

“The choice of freezing the first two layers probably had the effect to freeze those layers that are associated with the learning of “simple” patterns in images, such as horizontal and vertical borders, diagonals and so on. These kinds of patterns can be found in almost all images, so the fact that ResNet-18 was not pretrained on GTSRB likely did not significantly impact the ability of these early layers to extract useful low-level features.”

Finally the previous MLP-head FineTuned resnet was again employed to evaluate results. It was actually trained on half of the epochs of the linear. Silhouette’s coefficient with MLP was 0.82 and Linear 0.87. About this result (that is influenced by the restricted number of epochs) a personal opinion (that should be verified) is that: I personally do not expect the Silhouette’s coefficient to improve too much, because somehow the Linear layer is forced to linearly separate data, this could probably lead to representations that are closer in the space, while MLP, by using non linear transformations could not require compactness between equally labeled data.

Laboratory 2: The Transformative Transformer

Laboratory 2 is about the usage of HuggingFace ecosystem for retrieving datasets, models and pipelines. In the first part of exercise 1 some additional code has been developed in order to assess the rotten tomatoes dataset composition. Dataset labels are binaries namely “positive” and “negative” in reference to the strings values X, which represent films reviews from the specialized website Rotten Tomatoes. This “Exploratory Data Analysis” was not that deep, for each data split the minimum text length, the maximum text length, the mean length and the count of positive and negatives were computed. Probably it is not worth it to share here those results (that are accessible in the folder data stats).

In the second part of exercise 1 AutoModel and AutoTokenizer have been used in order to retrieve both model and tokenizer from HuggingFace via the model slug. A class CustomTokenizer (located in utils) have been developed for the following reasons:

- take a bit of confidence with these kind of objects,
- I was actually struggling with the legacy code provided via flipped lecture (to be honest I’m not sure what was the reason yet, but searching also in HF blogs it seems that there are differences in Rust implementations of Tokenizer and models in general, these kind of implementations are named “FAST”, such as: FastTokenizer and so on and I’m not sure why I ended up with something different from the legacy code... by the way it was interesting also because it was noticed that there are some differences in calling the Tokenizer as a functor or via class methods)
- Collect statistics about tokens and inherited vocabulary: in this sense the idea was to extrapolate the vocabulary of the tokenizer (which is the same of the model) and, since this model is part of the BERT family, search if there were some tokens in the Rotten Tomatoes datasets that were not in the vocabulary (i.e: that would have been marked with special token [UNK]), major details are posted in code comments in file LAB2_exercise1_2.py. Also more interesting statistics about lengths were computed in this phase: min/max/mean lengths of tokens in datasets. Despite this information wasn’t actually used throughout the rest of the exercise (it wasn’t needed), it could be considered as “security measure” as the stratification check applied in Laboratory 1. The reason is that, since we know that Transformers, in order to properly apply the attention mechanism, need to work on equal length tokens, what these models (AutoModels etc...) require, is the presence of the padding special token. By considering in advantage statistics about tokens, in case of future “strange behaviors” that would have been difficult to understand these stats could have been a starting analysis point (This statistics can be found in data_stats/training_set_TOKENS.json, validation_set_TOKENS.json and test_set_TOKENS.json).

In the final part of exercise 1 (in file LAB2_exercise1_3.py) Distil BERT was employed as a features extractor. For this reason several functions in utils/helpers_functions.py have been written in order

to extract the class token (this part is emphasized in the Laboratory2 ReadMe and would probably be a repetition here).

Similarly to Laboratory 1 these embeddings were fed into an SVC that has served as baseline, here Optuna was employed to select the best SVC on several hyperparameters, that were chosen by taking a look at SciKit Learn doc (the code shows relative comments on this). It is important to highlight that probably it would not be the case to use Optuna for fine tuning what would have used some moments later as a baseline... this can be considered a personal curiosity.

A more suitable usage of Optuna would have been in exercise 2 (FineTuning DistilBERT on Rotten Tomatoes). But since in this case the Trainer class was used (I struggled in the usage of this class to be honest as you will read in the ReadMe and heavily relied on HuggingFace tutorials and also asked advices to Gemini), what has been done was to implement a from scratch class implementation of HyperparamSearcher (in utils/HyperparamOptim.py). Results and comments on this exercises are showed in this extract from the ReadME:

FineTuned Distil-BERT performances::

Class / Metric	Precision	Recall	F1-Score	Support
0	0.85	0.82	0.83	533
1	0.83	0.85	0.84	533
Accuracy	-	-	0.84	1066
Macro Avg	0.84	0.84	0.84	1066
Weighted Avg	0.84	0.84	0.84	1066

The overall accuracy of *FineTuned Distil-BERT* wrt to the *Baseline* is **+4%**, to be honest I would expect a more substantial improvement, but considering that the Distil-BERT model was already pretrained on a huge amount of text, it is quite reasonable to think that, the only feature extractor, was enough to get good representations. From this point of view a **+4%** can be seen as a solid result. Also we should consider that, while in the baseline the F1 GAP between positives and negatives was about 0.02, now it is in 0.01, this means that the fine tuned model is more balanced in assigning Positive and negatives labels.

The chosen exercise here is about fine tuning a CLIP model, the reason is that once again this is very interesting from a “representational” point of view and I personally found Vision Transformers, CLIP very interesting. This the only exercise of Lab 2 that has been deployed as a notebook (you can find it in notebooks/LAB2_exercise3.ipynb).

Several comments can be found both in the Lab2 Readme and in the notebook itself, I’m going to summarize part of the content here to provide some context.

Vision Transformers (ViTs) apply the attention mechanism to visual inputs, CLIP-like models tries to align in a shared representational space ViTs representations and Textual representations and this is achieved essentially with InfoNCE loss inducing the model to represent in its latent space related inputs close.

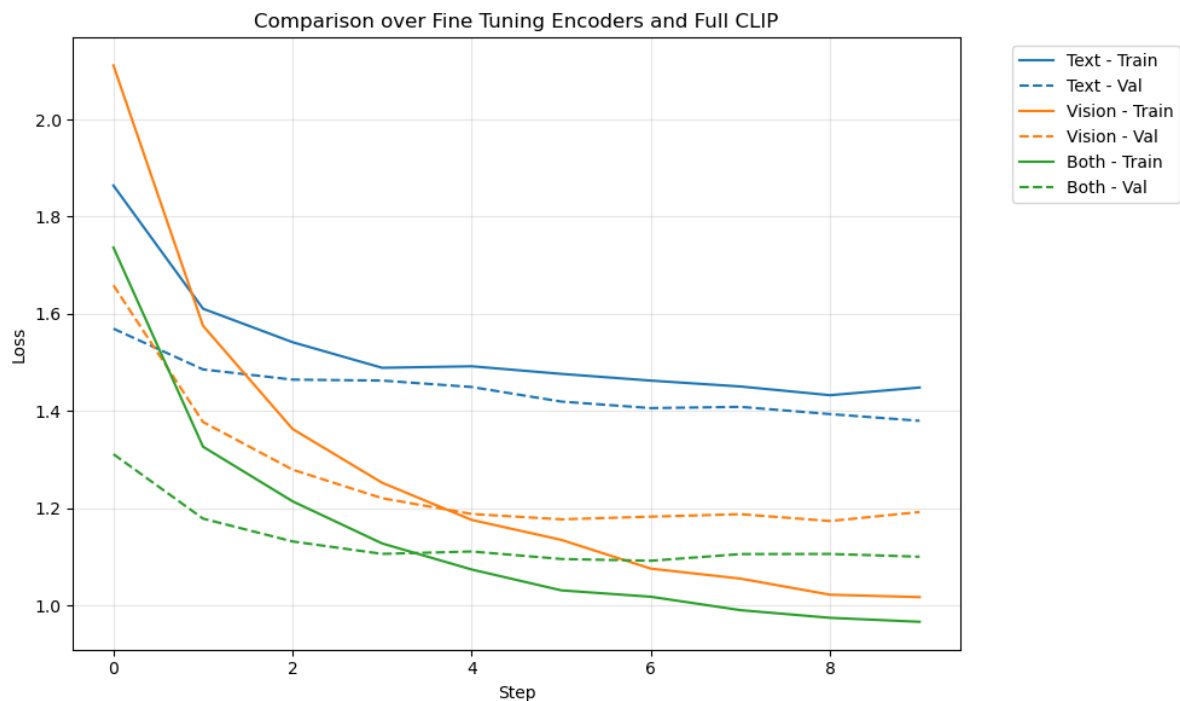
These models are trained on a huge amount of data, in fact a requirement was to try to work on different datasets. For this reason the CLIP model was FineTuned over tobacco3482 dataset, a dataset representing Documents Images regarding tobacco industry. Visual inputs are Document Images and Textual inputs are the type of documents those inputs represent.

In order to evaluate performances several experiments were conducted:

1. CLIP zero shot performances on tobacco 3482
2. CLIP PEFT-Fine Tuning (LoRA) over Text Encoder model
3. CLIP PEFT-Fine Tuning (LoRA) over Vision Encoder model
4. CLIP PEFT-Fine Tuning (LoRA) over both Text and Vision Encoder model

Comments analysis and results are detailed accross the notebook and posting them here would probably be redundant.

ps: there is a problem in rendering images on GitHub in the notebook (or more likely I didn't understood how to do that), you can find image in the directory imgs. The following image is a comparison of loss trends in different scenarios (2,3 and 4 of the aforementioned list):



Laboratory 3: Getting Up to Speed with Reinforcement Learning

In this last laboratory (of the chosen 3) the goal was to explore **gymnasium/farama** environment for Reinforcement Learning.

Gymnasium/farama offers some sandbox environments in which DRL algorithms can be tested. Legacy code (flipped lecture code) was about CartPole.

Each **environment** is built upon an **Observation Space**, in which **Observation Space Variables** lie. Since RL is based on **episodes** and each episode is composed of a series of **steps**, a point of view that helped me in understanding the first part of this laboratory, has been to imagine **each step** as a **realization** of the **Random Variables** lying in the **Observation Space**.

This point of view is in my opinion helpful in understanding the dynamics of this kind of learning:

1. the agent is in a certain **starting state** (i.e: a realization of each random variable in the observation space)
2. the **agent choose an action** to run (how it chooses that action depends on the policy)
3. the **action is applied** by the agent, this leads to **another state** (a different realization of the random variable) and contextually the environment provides a **reward** to the agent itself

The set of possible actions in Gymnasium is defined in the **action space**.

For the first exercise, not many variations have been implemented with respect to legacy code. The **main change** regards the usage of an **additional argument** (by default set as 1) which is

named **SCALE_FACTOR**. **SCALE_FACTOR** does not change anything, neither in the training phase, neither in the validation phase; it has been introduced as a “safety guard monitor”. The idea here is that (based on what was written above) every random variable in the Observation Space can be monitored (and must be), Gymnasium environments stops episodes when some of these random variables assume values that are not allowed, for instance in the CartPole environment we don’t want the pole to be on the ground, this means that when the angle between the cart and pole reaches a certain limit the “game is over”. **SCALE_FACTOR** allows us to set a kind of “**alert**” for each of these random variable: considered the original domain of each random variable if, during the training, its realization is somehow close to the “**fatal value**” that would terminate the training we can monitor it. The interpretation here is that we can take a look during the training how many times the “fatal value” was almost reached. The “almost” is defined by the user: ideally **SCALE_FACTOR** lies in (0, 1]; 1 is the standard exec, when **SCALE_FACTOR** ==1 the episode ends (that’s why 1 is default value); anything that lies into (0,1) is an actual **SCALE_FACTOR**.

In order to visualize this let’s consider the CartPole env description in Gymnasium (https://gymnasium.farama.org/environments/classic_control/cart_pole/); for this example we are going to take the Cart position random variable:

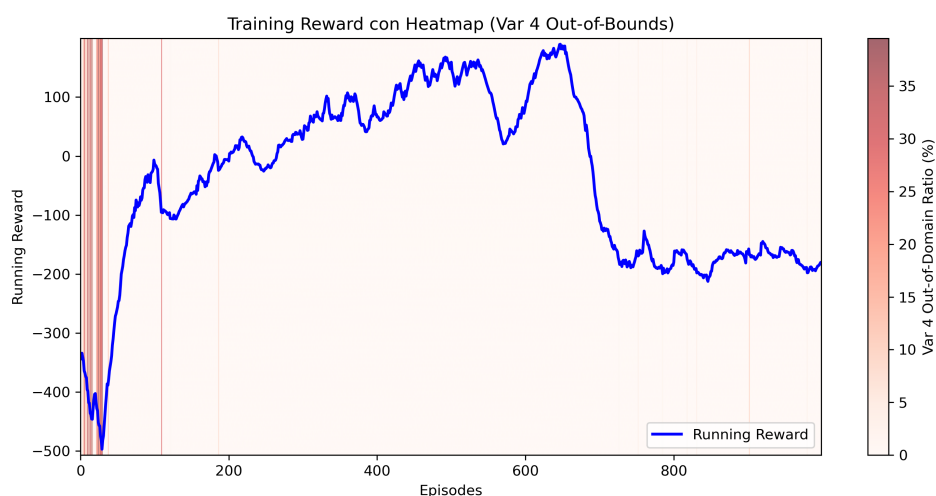
“The cart x-position (index 0) can take values between (-4.8, 4.8), but the episode terminates if the cart leaves the (-2.4, 2.4) range.”

if we set a scale factor of 0.5, we are asking the system to tell us when the cart reaches $x == -1.2$ or $x == 1.2$.

Is this actually useful? not if we hope to change the training, but I think it could provide some insights on **how the agent is learning** during the training. Also, *one could decide to change rewards form based on this kind of safety guards.*

In order to evaluate this kind of agent’s behavior a heatmap has been set as background of the rewards trend; an heatmap exists for each observation space variable (of course the rewards trend will be the same).

The following image shows an example of this: this is extracted from exercise 3 Lunars Lander v3 with A2C and MLPs Actor and Critic and seed 7 (several seeds were used during exercise 3). Var 4 in Lunars Lander v3 is linear velocity over y-axis. Var 4 has a domain of [-10, 10] and the scale factor was set to 0.45, this means that every time that the agent reached $-10 \times 0.45 = -0.45$ OR $10 \times 0.45 = 4.5$ we get an alert. In order to fully interpret the plot, consider that x-axis shows the Episodes and an episode is composed of several steps. The darker is the color, a greater number of steps, in that episode, the agent reached one of those alert values.



scale factor 0.45 alert for y-velocity in Lunar Lander with MLP A2C

Before moving over it worth it to remind how the observation space of both CartPole and Lunars Lander is made:

Num	Observation	Min	Max
0	Cart Position	-4.8	4.8
1	Cart Velocity	-Inf	Inf
2	Pole Angle	~ -0.418 rad (-24°)	~ 0.418 rad (24°)
3	Pole Angular Velocity	-Inf	Inf

CartPole observation space and domains

The state is an 8-dimensional vector: the coordinates of the lander in $\bar{x}$ & $\bar{y}$, its linear velocities in $\dot{\bar{x}}$ & $\dot{\bar{y}}$, its angle, its angular velocity, and two booleans that represent whether each leg is in contact with the ground or not.

```
Box([ -2.5 -2.5 -10. -10. -6.2831855 -10. -0. -0. ], [ 2.5 2.5 10. 10. 6.2831855 10. 1. 1. ], (8,), float32)
```

Lunars Lander observation space and domains

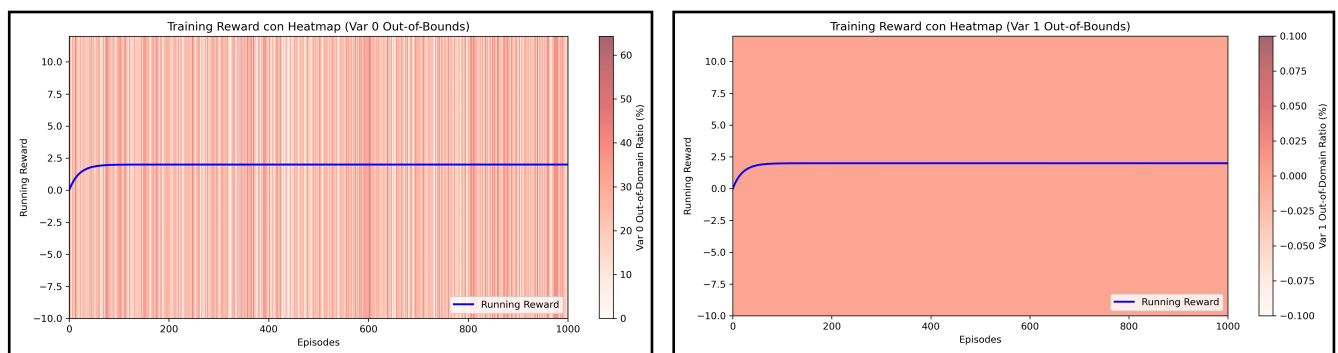
Regarding exercise 1 several experiments have been conducted using three different gammas $\gamma = [0.15, 0.5, 0.99]$ and two different scaling factor 0.02 and 0.75.

We are allowed to interpret this values as follows:

- **gamma** in the discount factor, the greater it is, the more the model will try to maximize future rewards. The smaller it is, the more the model is more attracted by a “greedy” approach (short terms rewards will have the priority over long terms).
- **scaling factor**: since Scaling Factor scales the domain of each random variable in the Observation Space, the greater it is, the larger the allowed margin is (i.e: less alerts will be notified).

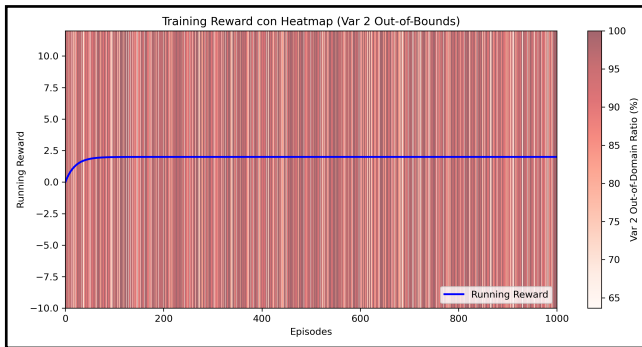
What follow are some comments of some interesting cases about exercise 1:

CARTPOLE RESULTS ON GAMMA == 0.5, SCALING FACTOR 0.02 (immediate rewards with restrictive margins):

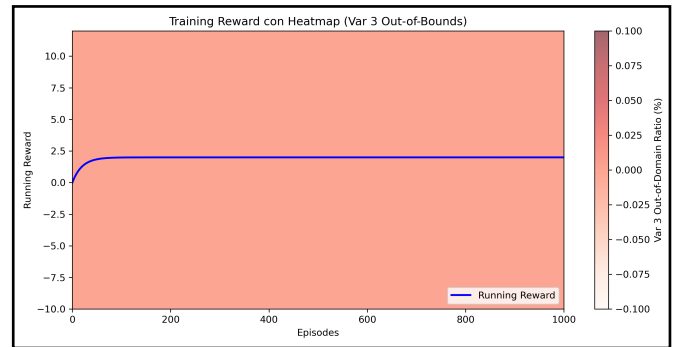


Rewards trend + Cart position

Rewards trend + velocity



Rewards trend + pole angle



Rewards trend + Pole Angular velocity

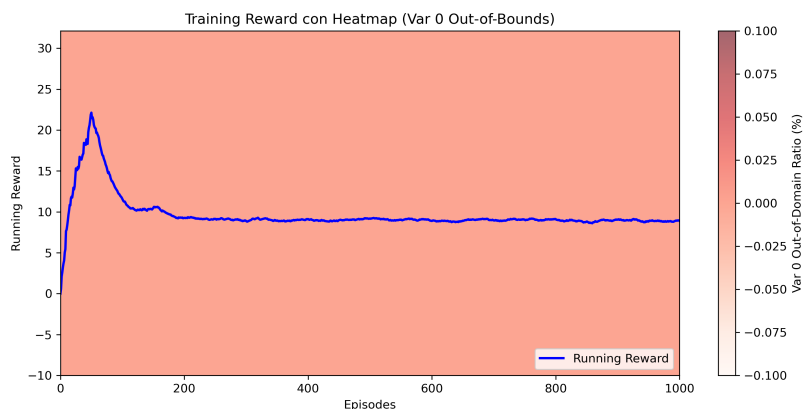
The first thing we are able to notice is that there is no learning: the blue line has an initial growth (rewards are growing), but gets flat very soon, this means that the model is not able to keep the pole in equilibrium and in very few steps it reaches the game over.

Background heatmaps show a fixed zero over **Cart velocity** and **Angular velocity** (their limits are infinite...); more interesting is the behavior of both **Cart position** and **Pole angle**. The last one has several very dark episodes, this means that in several steps the agent was **not able to maintain the pole in the scaled domain**, this is *actually fair* we are monitoring if the pole has move of $24 * 0.02 = 0.48$ grades from the standing position...

A similar conclusion can be applied to the **Cart position** $[-4.8*0.02, 4.8*0.02] = [-0.096, 0.096]$, (but since the game over is reached at 2.4 the fatal values become $[-0.048, 0.048]$).

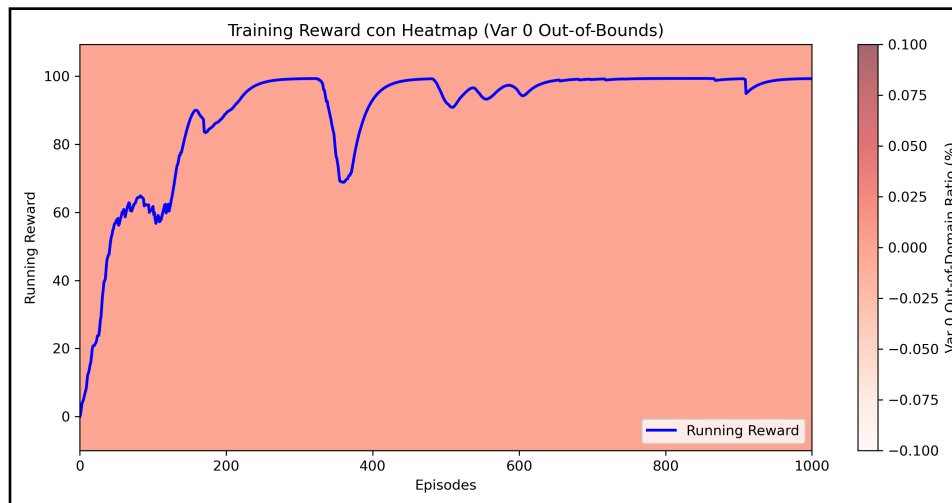
The next exercise was about improving the legacy reinforce algorithm adding a baseline. For this reason 3 main experiments have been conducted: Vanilla REINFORCE (no subtract), SUBTRACT and Value Net. In this exercise the SCALE_FACTOR was not considered (i.e: it was set as default to 1).

CartPole using Vanilla Reinforce: the reward is always positive for the whole episode, this means that the agent at the end will tend to maximize the probability of every action took in that episode, no matter if the action was actually a not good one.



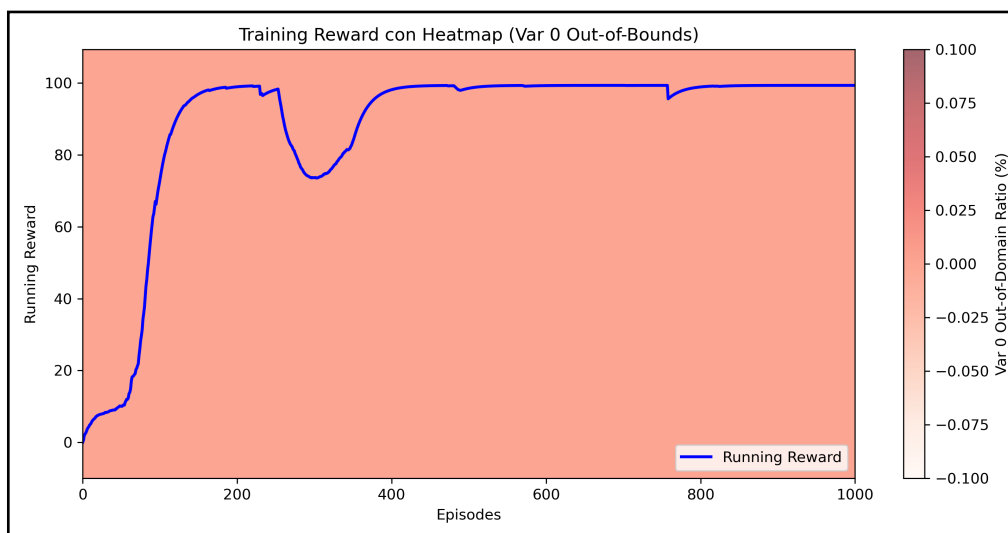
Vanilla Reinforce on CartPole

By subtracting the mean we are going to standardize all rewards within an episode. This means that if a single action “corrupted” the agent behavior, it will provide an under average result (i.e: its probability will not be improved):



subtracting REINFORCE on CartPole

In the final scenario, a Value Function was implemented to estimate the expected cumulative reward (return) of the agent in a given state. Under these conditions, the advantage is computed by subtracting the Value Function output from the actual return. If the result is positive, it means the action performed better than expected on average, increasing its selection probability; otherwise, if the advantage is negative, the action's probability is reduced.



Value Function Reinforce on CartPole

With both subtract and value function the agent reached a 500 of avg reward in validation phase.

Exercise 3: implementing Advantage Actor Critic (A2C) over CartPole and Lunars.

Seeds based Multiruns have been considered for this exercise. Seeds 42, 99 and 123 have been used in different scenarios.

total episodes: 1000

SCALE FACTOR CARTPOLE: 0.02 (very restrictive)

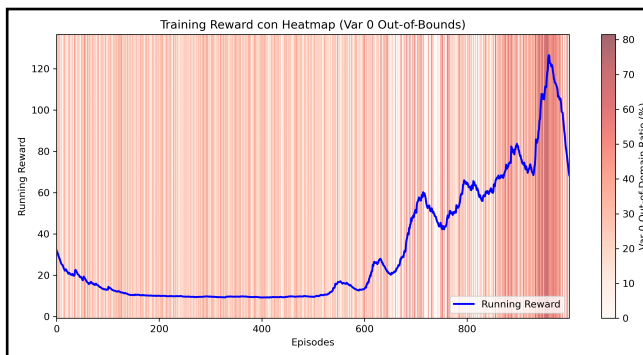
SCALE FACTOR LUNARS: 0.45 (wider margins)

Due to the first results obtained with Lunars applying Actor (policy net) and Critic (Value net) with a single hidden layer; another experiment has been conducted with “deeper” nets: 2 hidden layers.

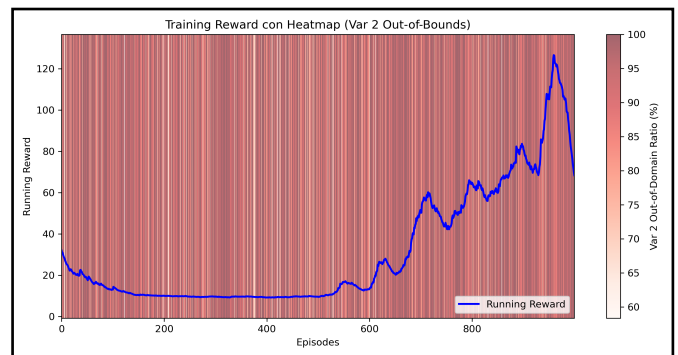
At the time this report is being written Lunars results are a tragedy and other experiments are probably going to be conducted.

What follow are some results obtained:

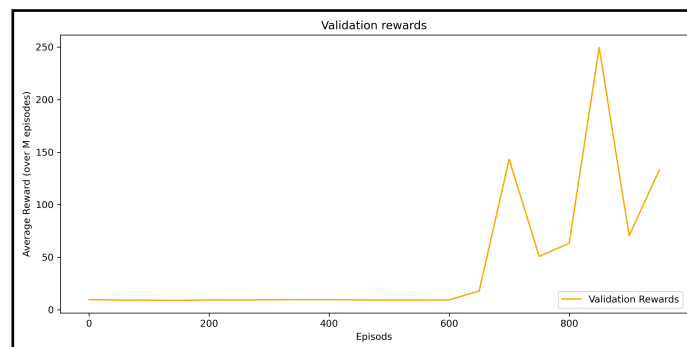
CARTPOLE A2C 1-hidden layer - seed 42 - SF: 0.02 (not considering Observation Variables with infinite domain)



Running rewards + Cartpole position



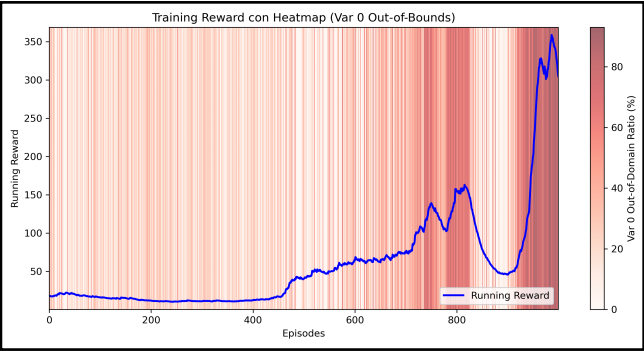
Running reward + Pole angle



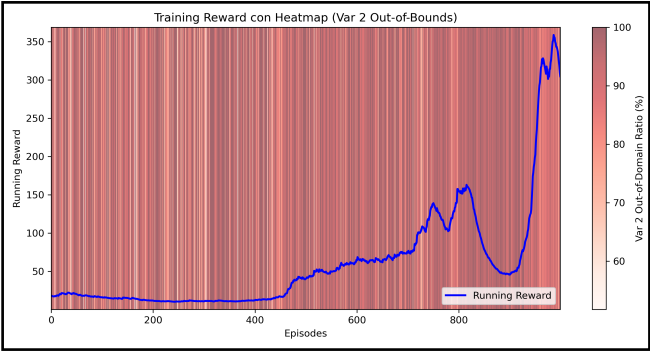
Validation phase running reward

Cartpole position plot offers some interesting insights: in the initial phase the heatmap shows that the number of triggers with respect to the scaled margin of cart position is lower than in the final part. Contextually in the final episodes, when the agent reaches its maximum rewards we can see a not-negligible amount of alerts. This shows a positive correlation between task performance and position alerts, demonstrating that the model learns to execute wider lateral movements with the cart to maintain the pole in balance.

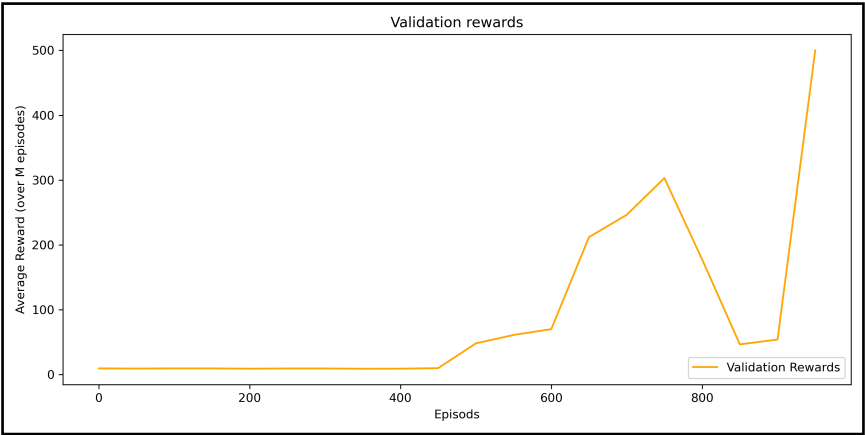
Using another seed (123):



Running reward + Cart position



Running rewards + Pole angle

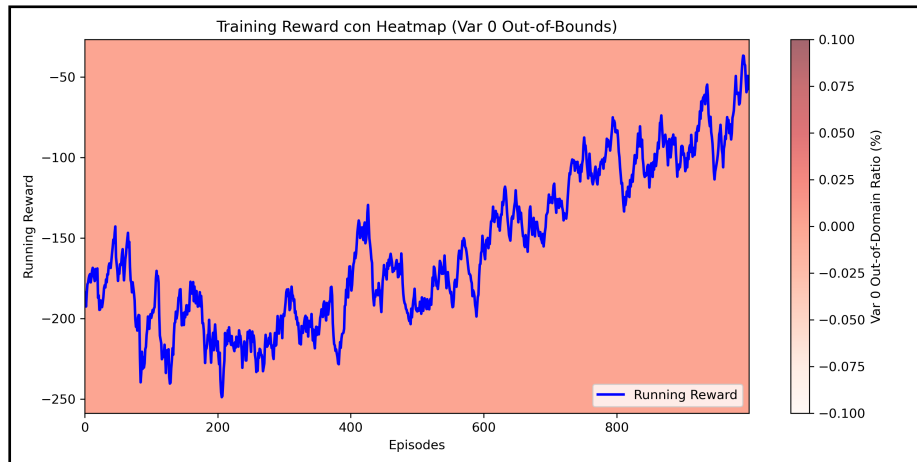


Validation phase running reward

LUNARS

Lunars performed very poorly on each configuration (several experiments also not registered here were made and in none occasion there were relevant results).

The following the Running reward on lunars using shallow MLP an both actor and critic.



An “improvement” however has been made by using the deeper MLP configuration:

